

HikeAlong

Version: 1.0

Supplementary Specification

Date: 25/May/26

Document Identifier: HikeAlong-1.0

Raisa Butuza

Group 30431

Revision History

Date	Version	Description	Author
25/May/26	1.0	Initial version of the Supplementary Specification for the HikeAlong event reservation system, including WebSocket notifications and UI/UX bonus features.	Raisa Butuza

Contents

1	Introduction	4
2	System Overview	5
2.1	Backend Overview	5
2.2	Frontend Overview	6
3	Bonus Features	7
3.1	WebSocket Reservation Notifications	7
3.2	UI/UX Improvements	7
4	Non-functional Requirements	8
4.1	Availability	8
4.2	Performance	9
4.3	Security	10
4.4	Testability	11
4.5	Usability	12
5	Design Constraints	14
5.1	Technology Constraints	14
5.2	Architectural Constraints	14
5.3	REST API Constraints	15
5.4	WebSocket Constraints	15
5.5	Frontend UI/UX Constraints	16
5.6	Search, Filter, and Sort Decision	16
5.7	Validation Constraints	17
5.8	Role-Based Constraints	17
5.9	Testing Constraints	18
6	Demo Scenarios	19
6.1	Scenario 1: User Makes a Reservation	19

6.2	Scenario 2: Organizer Updates Reservation Status	19
6.3	Scenario 3: UI/UX Validation	19
7	Conclusion	20

1. Introduction

This document represents the Supplementary Specification for HikeAlong, a full-stack event reservation system for hiking-related events. The system allows users to browse events, reserve seats, and receive updates about their reservations. Organizers can create and manage events, view reservations, and update reservation statuses.

The purpose of this document is to describe the non-functional requirements and design constraints of the system. These requirements are not always visible in the main use cases, but they are important for the quality, usability, maintainability, and reliability of the final application.

The application is implemented using:

- **Spring Boot** for the backend;
- **Angular** for the frontend;
- **Spring Data JPA** for persistence;
- **REST APIs** for standard CRUD communication;
- **WebSocket notifications** as a bonus feature for real-time reservation updates;
- **Angular Material** components for a cleaner and more consistent user interface.

The main entities of the system are:

- **Person**: stores user information such as name, email, password, age, and role;
- **Event**: stores hiking event information such as title, description, location, date, maximum number of participants, and organizer;
- **Reservation**: connects a person to an event and stores the number of reserved spots and the reservation status.

The supported user roles are:

- **USER**: can browse events, search and filter events, make reservations, and view reservation updates;
- **ORGANIZER**: can create, update, and delete events, view reservations for their events, and accept or reject reservation requests.

A major bonus part of the project is the real-time notification mechanism. When an organizer changes the status of a reservation, the backend can send a WebSocket notification. The frontend displays this notification using a toast-style UI component, so the user receives immediate feedback without manually refreshing the page.

Another important bonus part is the improved UI/UX. The frontend is not only a set of basic forms, but a role-based interface with navigation, event cards, reservation actions, filters, sorting, validation messages, and toast notifications.

2. System Overview

2.1 Backend Overview

The backend is structured using a layered architecture:

- **Controller layer:** exposes REST endpoints for login, persons, events, and reservations;
- **Service layer:** contains the main business logic;
- **Repository layer:** communicates with the database using Spring Data JPA;
- **Exception handling layer:** converts validation and business errors into meaningful HTTP responses;
- **WebSocket notification layer:** sends real-time updates when reservation statuses change.

The backend includes endpoints for:

- user login;
- person management;
- event management;
- reservation management;
- reservation status updates;
- WebSocket notification delivery.

2.2 Frontend Overview

The frontend is built using Angular and provides different screens depending on the authenticated user's role.

The main frontend pages are:

- **Login page:** allows users to authenticate;
- **Event feed:** displays available events using a user-friendly card layout;
- **Event details:** shows more information about a selected event;
- **Reservation flow:** allows users to reserve seats;
- **Organizer events page:** allows organizers to manage their own events;
- **Organizer reservations page:** allows organizers to accept or reject reservation requests;
- **Profile page:** allows users to view or edit their own profile;
- **Notification toast:** displays success, error, and WebSocket messages.

The frontend improves usability through:

- role-based navigation;
- visible buttons only for allowed actions;
- form validation messages;
- event search;
- event filtering;
- event sorting;
- clean error feedback;
- real-time reservation notifications.

3. Bonus Features

3.1 WebSocket Reservation Notifications

The WebSocket notification feature improves the reservation flow by making status changes visible immediately.

The main scenario is:

1. A user creates a reservation for an event.
2. The reservation is initially stored with a status such as `PENDING`.
3. The organizer opens the reservation management page.
4. The organizer accepts or rejects the reservation.
5. The backend updates the reservation status.
6. The backend sends a WebSocket notification.
7. The frontend receives the message and displays a toast notification.

Example notification messages:

- `Your reservation was accepted.`
- `Your reservation was rejected.`
- `A reservation status has been updated.`

This feature improves the user experience because users do not need to refresh the page repeatedly to check if their reservation was accepted or rejected.

3.2 UI/UX Improvements

The UI/UX part is treated as an important bonus because the application is demonstrated live through the frontend.

The frontend was designed to make the application easier to understand and use:

- the toolbar changes based on the current user role;
- users see event browsing and reservation actions;

- organizers see event management and reservation approval actions;
- actions that are not allowed for a role are hidden from the interface;
- event cards make the feed easier to read;
- search, filtering, and sorting help users find relevant events faster;
- toast notifications provide fast feedback after actions;
- form validation prevents invalid data from being submitted;
- backend validation errors are shown in a user-friendly way.

4. Non-functional Requirements

This section defines the main quality attributes of the system using quality attribute scenarios.

4.1 Availability

Quality attribute definition	Availability represents the ability of the system to remain usable during normal operation, especially for event browsing, reservation creation, and notification delivery.
Source of stimulus	A user, organizer, Angular frontend component, or WebSocket client.
Stimulus	A user opens the event feed, makes a reservation, or waits for a reservation status notification. An organizer updates a reservation status.
Environment	The system is running normally, with the Angular frontend connected to the Spring Boot backend and the database available. The WebSocket connection may be active or temporarily disconnected.
Artifact	Angular frontend, Spring Boot REST controllers, service layer, database, and WebSocket notification component.
Response	The system processes normal REST requests and sends notifications when reservation statuses change. If the WebSocket connection is not available, the main REST functionality remains usable.

Response measure	• The event feed and reservation actions should remain available during normal execution. • REST operations should not depend on the WebSocket connection. • If notification delivery fails, the reservation update should still be saved in the database. • Users can still see the updated reservation status after refreshing or reopening the page.
Tactics	• Separate the REST reservation flow from the WebSocket notification flow. • Use global exception handling for controlled backend errors. • Keep notification delivery as an additional feature, not as a requirement for saving data. • Allow the frontend to reconnect or continue without real-time messages.

4.2 Performance

Quality attribute definition	Performance represents how fast the system responds to user actions and how quickly the frontend updates after backend changes.
Source of stimulus	A user browsing events, an organizer managing reservations, or a WebSocket notification event generated by the backend.
Stimulus	The user searches events, filters events, sorts events, makes a reservation, or receives a notification after the organizer updates a reservation status.
Environment	The system is running with a normal academic/demo dataset and several users performing common actions.
Artifact	Event feed, Angular filtering logic, REST endpoints, reservation service, database, and WebSocket notification service.

Response	The frontend updates quickly after user actions. Notifications are displayed shortly after the backend sends the message.
Response measure	• Event feed loading should complete in under 2 seconds for a normal demo dataset. • Search, sort, and filter actions should feel immediate on the frontend. • Reservation creation should complete in under 2 seconds in local execution. • WebSocket toast notifications should appear within a few seconds after the backend sends the status update.
Tactics	• Use frontend filtering and sorting for fast interaction on small demo datasets. • Avoid unnecessary page reloads after reservation actions. • Use toast notifications instead of forcing users to manually refresh. • Keep REST payloads simple and focused. • Keep WebSocket messages short and specific.

4.3 Security

Quality attribute definition	Security represents the system's ability to protect user data, restrict actions based on role, and prevent invalid or unauthorized operations.
Source of stimulus	A user, organizer, or client trying to access frontend pages, REST endpoints, or notification-related functionality.
Stimulus	The user logs in, accesses a role-based page, creates an event, creates a reservation, or updates a reservation status.
Environment	The system is running normally and receives requests from the Angular frontend or from API testing tools.

Artifact	Login endpoint, security service, role-based frontend navigation, REST controllers, service layer, DTO validation, and WebSocket notification flow.
Response	The system allows only valid actions for the current role. Invalid login attempts return an unauthorized response. Invalid input returns a controlled validation response.
Response measure	• Failed login returns HTTP 401. • Invalid form data returns HTTP 400. • Users cannot access organizer-only actions through the normal UI. • Organizers can manage only event-related flows intended for their role. • Notifications should not expose sensitive information such as passwords or internal database details.
Tactics	• Use a dedicated login endpoint. • Store and check user roles. • Redirect users based on role after login. • Hide unauthorized buttons and routes in the frontend. • Validate all user input with DTO validation. • Send only minimal notification content through WebSocket messages.

4.4 Testability

Quality attribute definition	Testability represents how easily the system can be verified using unit tests, integration tests, and manual frontend demonstration scenarios.
Source of stimulus	Developer or evaluator running backend tests and checking frontend behavior during the project presentation.

Stimulus	Unit tests and integration tests are executed. The evaluator also tests the UI by logging in, creating events, making reservations, and updating reservation statuses.
Environment	The backend runs with a test database for integration tests. The frontend runs in the browser for manual testing and demo scenarios.
Artifact	Person, Event, and Reservation controllers and services, validation logic, WebSocket notification logic, Angular pages, and notification toast component.
Response	The system behaves correctly for valid and invalid scenarios. The frontend displays the correct pages and messages for each role.
Response measure	• Service logic for Person, Event, and Reservation should be covered by unit tests. • Controllers should be covered by integration tests. • Tests should include both successful and failing scenarios. • Manual demo should show login, role-based pages, event filtering, reservation creation, reservation status update, and notification display.
Tactics	• Keep controllers simple and delegate logic to services. • Keep validation rules in DTOs. • Keep notification sending in a separate service or component. • Test reservation status updates separately from UI display. • Prepare clear demo data for users, organizers, events, and reservations.

4.5 Usability

Quality attribute definition	Usability represents how easy, clear, and pleasant the system is for users and organizers when performing their main tasks.
-------------------------------------	---

Source of stimulus	A basic user or organizer interacting with the Angular frontend.
Stimulus	The user logs in, browses events, searches for an event, filters by location or date, reserves seats, receives a notification, or manages events as an organizer.
Environment	The frontend is loaded in a browser and connected to the backend. The user is authenticated and has a known role.
Artifact	Login page, toolbar, event feed, event cards, search/filter controls, reservation form, organizer dashboard, reservation management page, and notification toast.
Response	The frontend shows only relevant actions, guides the user through forms, displays clear success/error messages, and provides real-time reservation feedback.
Response measure	• A user can reach the event feed immediately after login. • A user can search and filter events without navigating away from the feed. • A user can make a reservation from the event interface. • An organizer can accept or reject a reservation from the organizer reservation page. • A notification toast appears after important actions. • Validation errors are understandable and shown close to the user action.

Tactics	• Use role-based toolbar links. • Use Angular Material components for consistent UI. • Use cards and clear spacing for event display. • Use buttons with clear action names such as Reserve, Edit, Delete, Accept, and Reject. • Use toast notifications for success, error, and real-time messages. • Use search, sorting, and filtering for the Event entity. • Avoid showing actions that the current role cannot perform.
----------------	---

5. Design Constraints

5.1 Technology Constraints

The project must respect the technologies already selected for the application:

- The backend is implemented in Java using Spring Boot.
- The frontend is implemented in Angular.
- The database is relational and accessed through Spring Data JPA.
- The communication between frontend and backend is mainly done through REST APIs.
- WebSocket communication is used as a bonus feature for reservation notifications.
- The UI uses Angular Material components for toolbar, buttons, forms, tables, and visual feedback.

5.2 Architectural Constraints

The backend must follow a layered architecture:

- controllers expose REST endpoints;

- services contain business logic;
- repositories access the database;
- DTOs validate input data;
- the global exception handler returns clear error responses;
- the WebSocket notification component is separated from the main reservation persistence logic.

This separation is important because the system should still save reservation changes even if the notification feature fails temporarily.

5.3 REST API Constraints

The system uses REST endpoints for the main application operations.

Endpoint	Purpose
POST /login	Authenticates a user and returns the login result and role.
/person	Handles person/user CRUD operations.
/event	Handles event CRUD operations.
/reservation	Handles reservation CRUD operations and reservation status updates.

The API must return meaningful status codes and validation messages. It should not expose internal stack traces or database details to the frontend.

5.4 WebSocket Constraints

The WebSocket notification system has the following constraints:

- WebSocket messages are used only for notification delivery.
- Reservation creation and status update logic must remain REST-based.
- A failed WebSocket delivery must not cancel a successful reservation update.
- Notifications should contain simple and safe data.
- The frontend must display notifications using a toast component.
- The application must remain usable even if WebSocket is disconnected.

Example notification payload:

```
{
  "type": "RESERVATION_STATUS_UPDATED",
  "message": "Your reservation was accepted.",
  "reservationId": "...",
  "status": "ACCEPTED"
}
```

5.5 Frontend UI/UX Constraints

The frontend must be clear enough to support a live demo. The following UI/UX constraints apply:

- The toolbar must change based on the logged-in user's role.
- USER accounts should see event browsing and reservation actions.
- ORGANIZER accounts should see event management and reservation management actions.
- Forms must show validation errors.
- The event feed must support searching, ordering, and filtering.
- At least two event filtering criteria should be available, such as location and date.
- Important actions must trigger feedback messages.
- WebSocket notifications must appear as toast messages.
- The interface should avoid clutter and should not show unavailable actions.

5.6 Search, Filter, and Sort Decision

The Event entity is selected for searching, filtering, and sorting because it is the main entity browsed by users. This also improves the live demo because the evaluator can immediately see that the frontend is not only a static CRUD interface.

The Event feed supports:

- search by title;
- filter by location;

- filter by upcoming date or date range;
- sort by date;
- optionally sort by number of participants or availability.

For this project, frontend filtering is acceptable because the demo dataset is small and it gives immediate feedback to the user. If the application grows, the same logic can be moved to the backend using repository queries.

5.7 Validation Constraints

The system must validate input before saving it.

- Person name must have a valid length.
- Email must have a valid format.
- Email must be unique.
- Password must be strong enough.
- Event title must be valid.
- Event location must be valid.
- Event date must be in the future.
- Maximum number of participants must be at least 1.
- Reserved spots must be at least 1.
- Reservation status must be valid.

5.8 Role-Based Constraints

The application must separate functionality by role.

- A USER can browse events and create reservations.
- A USER can receive reservation status notifications.
- An ORGANIZER can create, update, and delete events.

- An ORGANIZER can view reservations for events and update reservation statuses.
- The frontend should hide buttons that do not apply to the current role.
- The backend should still validate important business rules because frontend hiding is not enough for security.

5.9 Testing Constraints

The backend must be testable and should include meaningful tests for the most important flows.

The most important tested flows are:

- Person creation, update, duplicate email, and invalid data;
- Event creation, update, deletion, and invalid event data;
- Reservation creation, duplicate reservation, invalid number of spots, and status update;
- Controller integration tests for Person, Event, and Reservation;
- WebSocket notification trigger after reservation status update, if included in the test scope.

The frontend should also be manually demonstrable through the following scenarios:

- login as USER;
- browse event feed;
- search and filter events;
- create a reservation;
- login as ORGANIZER;
- accept or reject reservation;
- return to USER view and observe the notification or updated reservation status.

6. Demo Scenarios

6.1 Scenario 1: User Makes a Reservation

1. The user logs in.
2. The user is redirected to the event feed.
3. The user searches for a hiking event.
4. The user filters events by location or date.
5. The user selects an event.
6. The user reserves one or more spots.
7. The frontend displays a success toast.
8. The reservation is saved with status `PENDING`.

6.2 Scenario 2: Organizer Updates Reservation Status

1. The organizer logs in.
2. The organizer opens the reservations page.
3. The organizer sees pending reservations for their events.
4. The organizer accepts or rejects a reservation.
5. The backend updates the reservation status.
6. The backend sends a WebSocket notification.
7. The user receives a toast message.

6.3 Scenario 3: UI/UX Validation

1. The user tries to submit an invalid form.
2. The frontend or backend validation catches the error.
3. A clear error message is displayed.
4. The user corrects the input.
5. The action succeeds and a success message is shown.

7. Conclusion

The HikeAlongsupplementary specification describes the main quality requirements and design constraints of the application. Besides the required CRUD, authentication, validation, and testing flows, the project also includes bonus features that improve the final product.

The most important bonus features are:

- real-time WebSocket notifications for reservation status changes;
- toast notifications in the frontend;
- role-based navigation;
- improved event feed UI;
- search, filtering, and sorting for events;
- clear validation and error messages;
- separate user and organizer experiences.

These additions make the project stronger because they improve both the technical design and the user experience. The WebSocket feature shows that the system can support real-time behavior, while the UI/UX improvements make the application easier to use and better prepared for a live demonstration.